

Top Trader Flash

Public Binary API Documentation

Version 1.0.12

Revision History

Date	Version	By	Description				
24 Apr 2014		Ferry Kong	First Edition				
28 Apr 2014	1.0.2	Ferry Kong	Remove trailer on message packet (page. 6)				
2 May 2014	1.0.3	Ferry Kong	Add the following API E_ORDERS_LISTALL E_ORDERS_FILL_LISTALL E_CASH_BALANCE E_INSTRUMENT_BALANCE				
8 May 2014	1.0.4	Ferry Kong	All error code to the error reply message (page 8, 10, 15, 18, 21) Add named pipe connection (page 5)				
28 May 2014	1.0.5	Ferry Kong	Add E_ORDERS_OUTSTANDING_LISTALL API (page 28) Add E_HEARTBEAT API (page 31)				
3 Jun 2014	1.0.6	Ferry Kong	Add Stock Code and Side to reply value of E_ORDERS_OUTSTANDING_LISTALL API (page 28) Add E_GET_TIME API (page 32)				
5 Jun 2014	1.0.7	Ferry Kong	Add Order Price to reply value of E_ORDERS_OUTSTANDING_LISTALL API (page 28)				
18 Jun 2014	1.0.8	Ferry Kong	Add Out-of-throttle-rate error code (page 10, 15, 17, 18, 21) Add System not ready error code (page 10, 15, 17, 18, 21)				
10 Jul 2014	1.0.9	Ferry Kong	Add Reference to reply value of E_ORDERS_OUTSTANDING_LISTALL API (page 28)				
10 Sep 2015	1.0.10	Ferry Kong	Add Resubmit flag to E_ORDERS_NEW API (page 14) Add named pipe definition (page 5) Add back the followings missing output fields of E_ORDERS_OUTSTANDING_LISTALL - Stock Code - Order Side - Reference (page 28) Description correction: <table><tr><th>Wrong description</th><th>Correct description</th></tr><tr><td>Order push report generated by the system set the reference to all zero. (page 6 and 7)</td><td>The last reference engaged with the order is supplied with order push report. (Page 6 and 7)</td></tr></table>	Wrong description	Correct description	Order push report generated by the system set the reference to all zero. (page 6 and 7)	The last reference engaged with the order is supplied with order push report. (Page 6 and 7)
Wrong description	Correct description						
Order push report generated by the system set the reference to all zero. (page 6 and 7)	The last reference engaged with the order is supplied with order push report. (Page 6 and 7)						
4 Nov 2016	1.0.11	Ferry Kong	Add Broker ID to E_ORDERS_NEW API (page 14)				
26 Jan 2017	1.0.12	Ferry Kong	Add description for use of library				

Table of Content

Revision History	2
Table of Content.....	3
Introduction.....	5
Connection	6
Data Packet.....	8
Session.....	9
Reference.....	9
Logon Mechanism.....	9
Binary API Return Format.....	10
Single Row Reply	10
Order Push	10
Multiple Row Reply.....	10
Error Reply.....	10
Order Report Message.....	11
Order Status.....	12
Error Code.....	12
Binary API Definition	13
E_LOGON – Logon to the system.....	13
E_LOGON_CHANGE_PIN – Change the logon password.....	15
E_ORDERS_NEW – Place a new simple order for HKEx	16
E_ORDERS_REQUEST_MODIFY – Modify an existing order	19
E_ORDERS_REQUEST_CANCEL – Cancel an existing order.....	22
E_ORDERS_REPORT_PUSH – Order report generate by system	25
E_ORDERS_REPORT_LIST – List out the order reports.....	27
E_ORDERS_LISTALL – List out all the orders	29
E_ORDERS_FILL_LISTALL – List out all the orders	30
E_ORDERS_OUTSTANDING_LISTALL – List out all outstanding orders	31
E_CASH_BALANCE – Get the cash balances	32
E_INSTRUMENT_BALANCE – Get the instrument balances	33
E_HEARTBEAT – Heart beat to system	34
E_GET_TIME – Get local time	35
Library function Definition.....	36
StartTopTraderFlash function.....	36
StopTopTraderFlash function.....	37
BindAccount function.....	38
SendClientRequest function.....	39

UnbindAccount function.....	40
TopTraderFlashAccountCallback function	41

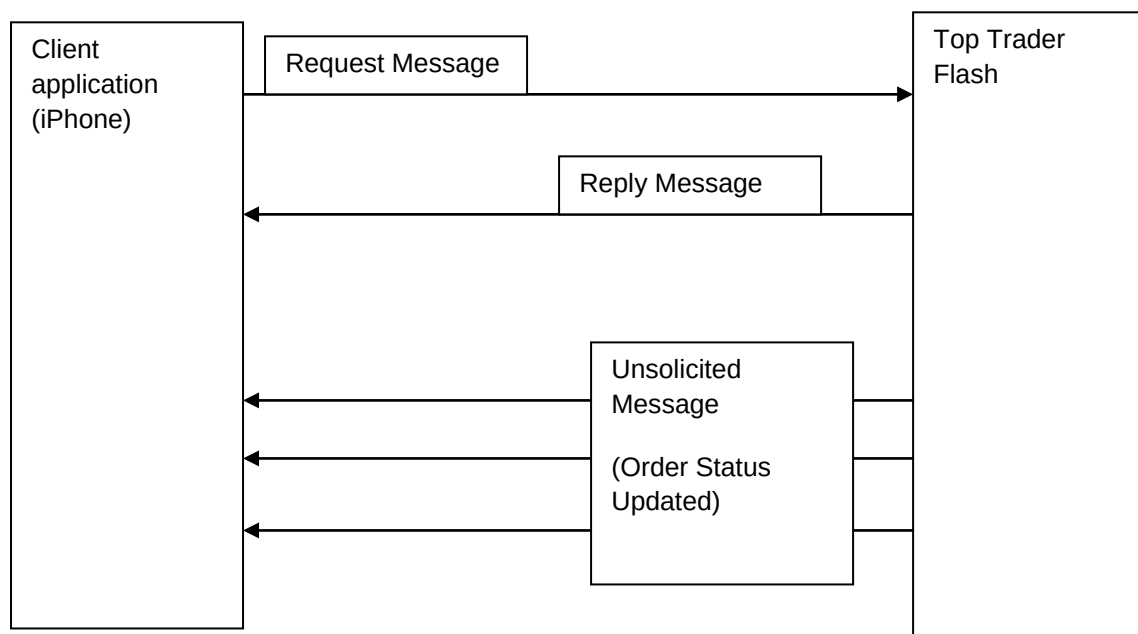
Introduction

Top Trader Flash provides a set of public API so that client application can be developed and connected to the system for service.

The following service are provided by the API

- Client authentication
- Place order
- Modify order price
- Reduce outstanding order quantity
- Cancel outstanding order
- Order enquiry

The client application sends request to the API and API reply to the application asynchronous. And API will send unsolicited message to the client application to notify the order status change or client balance change, etc...



Connection

System supports three types of connections.

- For Windows platform, both TCP/IP and named pipe connection are supported.
- For Linux platform, only TCP/IP connection is supported.
- For both Windows and Linux platform, TopTraderFlash is compiled as a library. Client application load the library onto its process.

TCP/IP connection

API caller shall make a TCP/IP to the pre-defined port of the system. The client requests service by sending predefined binary data to the server. And the binary data shall be packaged with the predefined data packet layout. The server replies a predefined data packet with binary data back to the client. The request and reply of the message is in asynchronous mode. Once successfully authenticated, the server will provide the client with the unique session code. All subsequent requests to server must bear the same session code. When making request to the server, the client application shall provide the session code and generate a unique reference code to the server. When server reply message to the client asynchronously, the client's request reference code will be returned in the message header, so that client knows which request the server replies to.

If the TCP connection is broken, the client has to make a fresh connection with the server and re-authenticate with server again. The server will provide a new session code to the client. All subsequent requests to the server must use the newly provided session code.

Library

A sets of functions are exported by the library. API caller shall load the library onto its process.

Instead of sending packed binary data via TCP, caller call library function, passing-in the packed binary data as parameter.

The binary data shall be packaged with the predefined data packet layout. The return value of function indicates if the function request is success or not. The result will then be returned to the caller by involving a callback function provided by caller. The parameter of callback function will be filled-in with a predefined data packet. Basically, the request and reply is in asynchronous mode.

Once successfully authenticated, the server will provide the client with the unique session code. All subsequent requests to server must bear the same session code. When making request to the server, the client application shall provide the session code and generate a unique reference code to the server. When library reply message to the client asynchronously, the client's request reference code will be returned in the message header, so that client knows which request the server replies to.

Named Pipe connection

Named pipe connection is supported on Windows platform only.

System pre-defines one receiving named pipe channel and one sending named pipe channel for the API caller.

API caller shall make two separate named pipe connection to the system, one for sending message to the system; and one for receiving message.

API caller can only send message to the system via the receiving named pipe channel; and shall listen to the system's sending named pipe channel.

The client requests service by sending predefined binary data to the server. And the binary data shall be packaged with the predefined data packet layout. The server replies a predefined data packet with binary data back to the client. The request and reply of the message is in asynchronous mode. Once successfully authenticated, the server will provide the client with the unique session code. All subsequent requests to server must bear the same session code. When making request to the server, the client application shall provide the session code and generate a unique reference code to the server. When server reply message to the client asynchronously, the client's request reference code will be returned in the message header, so that client knows which request the server replies to.

BSS Receiving Named Pipe:

\\.\pipe\TopHFPipeRead

BSS Sending Named Pipe:

\\.\pipe\TopHFPipeSend

Data Packet

Binary data is sent and received between client and server must be packed with header.

A header, which is 32 bytes long, must be inserted in front of the data. The header includes information about the length of the data and type of data. Therefore, entire data packet includes header and the actual.

The server will send data to client in one complete data packet. However, the data arrive to the client application may be chunked into many smaller network packet due to the nature of TCP/IP networking. Sometimes, one network packet may include more than one data packet from the server. In general, one network package received by client application may contain part of data package, one data package, more than one data packet; and part of the remaining data package. Client shall use the packet layout to reassemble each complete data packet.

Binary Data is in Little-Endian format.

Packet Layout

Packet Header (32 bytes)	Actual Data (various length)
-----------------------------	---------------------------------

Packet Header Layout

Field Name	Description	Number of Bytes	Value
STX	Packet Header	1 byte	0x01
Total Data Length	The number of bytes including header and actual data	4 bytes	For example, if data length is 165 bytes, the bytes shall be 0xA5, 0x00, 0x00, 0x00.
API Type Flag	Indicate type of message	1 byte	0x54 = API request 0x55 = API reply with single row data 0x56 = API reply with multiple rows data 0x57 = Order Push Report (single row only) 0x58 = API error reply
API ID	An unique API ID for each message	4 bytes	Refer to API definition
Session ID	A session ID generates by system after logon	8 bytes	Random value. It is 8 bytes, and NOT a null terminated string.
Reference	Reference field pass-in by API caller.	14 bytes	For API reply, system put back the caller's API reference. For order push report, reference field contains the last reference engaged with the order is supplied.

Session

The session is unique sequence of bytes (8 bytes) generated by the system upon receive of logon request from client. It is for authenticate each client's subsequent API request after successful logon. All request messages, except for the first logon request, must bear this unique session code.

Reference

The reference is a 14 bytes field and is provided by the API caller, which the system will be returned on each reply message. It is aimed to let the client to associate the reply message with the request message. It is mandated that the reference provided by client is unique for each session.

Logon Mechanism

Caller shall make a logon request to the system. The Session field (8 bytes) on the header shall be set to zero (0x00). Caller shall set the Reference (14 bytes) field on the header. If logon success, the Session field on the reply message header contains the Session ID(8 bytes), all subsequent API request must use this Session ID, otherwise, an error reply message with reason "Invalid session" returns; and system close the connection to the caller.

If logon is not success, system closes the connection to the caller.

Binary API Return Format

There are four kinds of API return formats,

- Single row reply
- Order push
- Multiple row reply
- Error reply

Single Row Reply

Only contains fields on the API document.

Single Row Data
Refer to API document for each reply fields definition

Order Push

Only contains fields on the API document.

Order Push Data
Refer to API document for order push fields definition

Multiple Row Reply

A count (4 bytes) is added before the data to indicate number of rows contains. Each row is in same length.

Count	Multiple Rows			
4 bytes (no of row)	1 st row	2 nd row	3 rd row	4 th row and

Error Reply

The first 4 bytes is the error code; and the subsequent 28 bytes are reserved. An ASCII null terminated string is appended starting from 33th byte. The string contains a description of the error in English. The possible error code is defined on each API.

Error Code	Reserved	Error Reason
4 bytes Integer error code	28 bytes reserved	An ASCII null terminated string contains the error description in English

Order Report Message

System generates order report message when the following happens:

- In response to new order request
- In response to order modify request
- In response to order cancel request
- In receive order cancellation message from HKEx
- In receive order filled message from HKEx
- In receive trade cancel message from HKEx

The order report message is fixed length. The Report Type field on the message indicates what type of message.

The following order push message will be sent for change of order status.

Order Report Type	Code	Description
Order Report	0x00	Order is queued or order is rejected
Order Fill	0x01	Order fill is reported by HKEx
Order Modify Approve	0x02	Order modify request is approved
Order Modify Reject	0x03	Order modify request is rejected
Order Cancel Approve	0x04	Order cancel request is approved
Order Cancel Reject	0x05	Order cancel request is rejected
Unsolicited Order Cancel	0x06	Order cancelled by HKEx
Trade Cancel	0x07	Execution (Trade) is cancelled by HKEx

Order Status

The order status of each order indicates the state of the order. It also governs what further action can be done on the order.

With suitable order status, user/client can:

- Request to cancel
- Request to modify

The following table summarizes all order status and possible further action.

Status	Description	Can request modify?	Can request cancel?
301	Order queued on Exchange	Y	Y
401	Order is rejected by Exchange or Credit Officer	N	N
402	Order is cancelled due to user/client request	N	N
403	Order is deleted/removed by Exchange	N	N
601	Order is partially filled	Y	Y
801	Order is fully filled	N	N
999	Unknown status. It is an error state	N	N

Error Code

The error reply is generated by the system if the pre-checking condition is not meet. For example, insufficient fund or cannot find the order.

After that, all negative reply from HKEx will be replied to API caller in form of Order Report; and the Text field of the Order Report will contain the text returned by HKEx. For information, HKEx does not provide error code for her error message.

If the error is generated by the system, the first 4 bytes after the API error reply header contains a 32-bit integer, representing the error code. Each API defines the possible error code returned by that API.

The following is a list of all possible error codes generated by the system.

Error Reply

Error Code	Description
61001	Unauthorized access. The requesting API is made to a connection that does not logon successfully; or does not perform logon at all.
61002	Invalid session. The session on the requesting API header is not matched.
81001	Insufficient fund when placing buy order or increasing the buy order quantity/price
81002	Insufficient stock balance when placing sell order or increasing sell order quantity
81003	Order not found. This error is generated if the system cannot locate the order base on the OrderRef of requesting API.
81004	Market closed
81005	Out of throttle rate. The error reason contains the milliseconds to wait for next available throttle
81006	System not ready

Binary API Definition

E_LOGON – Logon to the system

API ID Number

81102101

Description

Use this API to logon to the system

Request Parameters

Name	Start	Length	Data Type	Description	Values
AccountID	0	50	String	AccountID of the client	Null terminated string
PIN	50	200	String	The PIN. The pin which hashed in SHA256; and in HEX string format	Null terminated string

Reply

Single row API reply will be returned.

Reply Parameter

Name	Start	Length	Data Type	Description	Values (HEX)
Logon Result	0	4	32-bit integer	The result of the logon	0 = Success 1 = Wrong Pin 2 = Account Expired 3 = Account Locked 99 = Others

The PIN shall be hashed with SHA256 method.

For example, if the plain text of the PIN is “pass”, the SHA256 hashed value shall be “d74ff0ee8da3b9806b18c877dbf29bbde50b5bd8e4dad7a3a725000feb82e8f1”

If logon success, the API header will be filled with Session ID, all subsequent API call must use this Session ID.

E_LOGON_CHANGE_PIN – Change the logon password

API ID Number

81102102

Description

Use this API to logon to the system

Request Parameters

Name	Start	Length	Data Type	Description	Values
AccountID	0	50	String	AccountID of the client	Null terminated string
Old PIN	50	200	String	The PIN. The pin which hashed in SHA256; and in HEX string format	Null terminated string
New PIN	250	200	String	The PIN. The pin which hashed in SHA256; and in HEX string format	Null terminated string

Reply

Single row API reply will be returned.

Reply Parameter

Name	Start	Length	Data Type	Description	Values (HEX)
Result	0	4	32-bit integer	The result of the change password	0 = Success 1 = Wrong Old Pin

The PIN shall be hashed with SHA256 method.

For example, if the plain text of the PIN is “pass”, the SHA256 hashed value shall be “d74ff0ee8da3b9806b18c877dbf29bbde50b5bd8e4dad7a3a725000feb82e8f1”

E_ORDERS_NEW – Place a new simple order for HKEx**API ID Number**

81101101

Description

Use this API to place a new simple order for HKEx

Request Parameters

Name	Start	Length	Data Type	Description	Values (HEX)
Quantity	0	8	64-bit Integer #	Quantity	
Price	8	8	64-bit Integer #	Price	
Stock Code	16	4	32-bit integer	HKEx Numeric Stock code 1 = Cheung Kong 5 = HSBC	For example, 1 = 0x01, 0x00, 0x00, 0x00 (Little-endian)
Order Side	20	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	21	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	22	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Ignore Price Check	23	1	Byte	Instruct HKEx to ignore price warning checking	0x00 = HKEx perform checking 0x01 = HKEx ignore checking
BrokerID	24	4	32-bit integer	Desired submitting broker id. Zero if no specified broker id	

Signed 64 bit integer fields with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.

Remark

If there is no throttle rate available, an error reply will be returned;

Resubmit flag

All first time order submission must set this flag to 0x00. If that order is rejected by HKEx due to price warning, the caller can place a new order with same price and quantity with the ignore price check flag set to 0x01.

Please note that from both BSS and HKEx point of view, the first and second orders are two separate orders. Each order has its own OrderRef, ExecutionID, ReportID, etc.... The only different is that if flag is 0x01, BSS requests HKEx not to perform price warning checking.

If an order is rejected by HKEx due to price warning, an order report reply will be returned to caller.

Reply

If the request passes basic validation, risk checking, the following single row order report message will be returned. Otherwise, error reply will be returned. Possible error codes are listed below.

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer #	Order quantity	
Order Price	16	8	64-bit Integer #	Order price	
Order Outstanding Qty	24	8	64-bit Integer #	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer #	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer #	For Order Report, always zero	
Filled Price	48	8	64-bit Integer #	For Order Report, always zero	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Report Type	63	1	Byte	Indicator of this message	0x00 = Order Report
Broker ID	64	12	String	Broker ID for this order	Null terminated ASCII string
Transaction Date Time	76	25	String	The transaction date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	Always empty	null terminated ASCII string
Text	122	96	String	The reject reason	If order is rejected, contains a null terminated ASCII string reason
Report ID	218	4	32-bit Integer	This Report ID	

Error Reply

Error Code	Description
81001	Insufficient fund
81002	Insufficient stock balance
81005	Out of throttle rate. The error reason contains the milliseconds to wait for next throttle
81006	System not ready

New Order Example

Request data packet (HEX)

Qty: 2000
 Price: 122.1
 Stock Code: 00001
 Side: 1
 Order Type: 0x45 (EL)
 AON: 0
 Resubmit: 0
 Broker ID: 6001

```

01 38 00 00 00 54 2D 81 D5 04 50 32 6B 41 6C 4D
69 47 20 20 20 20 20 20 20 20 20 20 31 30 30 32
00 D0 ED 90 2E 00 00 00 80 D0 C5 D7 02 00 00 00
01 00 00 00 01 45 00 00 71 17 00 00
    
```

Reply data packet (HEX)

OrderRef: 300001
 Status: 301
 Order Qty: 2,000
 Order Price: 122.100
 Order OS Qty: 2,000
 Order Filled Qty: 0
 Filled Qty: 0
 Filled Price: .000
 Stock Code: 1
 Side: B
 AON: N
 Order Type: EL
 Message Type: Order
 Broker ID: 3032
 Txn Time: 20140425155020968
 Execution ID:
 Text:

```

01 FE 00 00 00 55 2D 81 D5 04 50 32 6B 41 6C 4D
69 47 20 20 20 20 20 20 20 20 20 20 31 30 30 32
E1 93 04 00 2D 01 00 00 00 D0 ED 90 2E 00 00 00
80 D0 C5 D7 02 00 00 00 00 D0 ED 90 2E 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 01 00 00 00 01 45 00 00
33 30 33 32 00 00 00 00 00 00 00 00 32 30 31 34
30 34 32 35 31 35 35 30 32 30 39 36 38 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
    
```

The values highlighted with color are API header and trailer.

E_ORDERS_REQUEST_MODIFY – Modify an existing order**API ID Number**

81101102

Description

Use this API to modify an existing order

Request Parameters

Name	Start	Length	Data Type	Description	Values (HEX)
OrderRef	0	4	32-bit integer	Order Ref of an order	
Quantity	4	8	64-bit Integer #	New quantity of entire order. If no change in price, pass-in the original quantity to this field	
Price	12	8	64-bit Integer #	The new price of the order. If no change in price, pass-in the original price to this field	

Signed encoding 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.

Remarks

The input parameter Qty represents the new quantity of entire order. For example, if client want to modify an existing order from original quantity 10,000 to 7,000; and there are already 2,000 filled, application shall still pass-in 7,000 to Qty field.

Caller can modify the price and reduce/increase the quantity at the same time.

Reply

If the request passes basic validation, risk checking, the following single row order report message will be returned. Otherwise, error reply will be returned.

Remark

If there is no throttle rate available, an error reply will be returned;

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer #	Order quantity	If modify is approved, it is new quantity. If modify is rejected, it is the previous quantity
Order Price	16	8	64-bit Integer #	Order price	If modify is approved, it is the new price. If modify is rejected, it is the previous price
Order Outstanding Qty	24	8	64-bit Integer #	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer #	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer #	Always zero	
Filled Price	48	8	64-bit Integer #	Always zero	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Report Type	63	1	Byte	Indicator of this message	0x02 = Order Modify Approve 0x03 = Order Modify Reject
Broker ID	64	12	String	Broker ID for this order	Null terminated ASCII string
Transaction Date Time	76	25	String	The transaction date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	Always empty	null terminated ASCII string
Text	122	96	String	The reject reason	If rejected, contains a null terminated ASCII string reason
Report ID	218	4	32-bit Integer	This report ID	

Error Reply

Error Code	Description
81001	Insufficient fund
81002	Insufficient stock balance
81003	Order not found
81005	Out of throttle rate. The error reason contains the milliseconds to wait for next throttle
81006	System not ready

Modify Order Example

Request data packet (HEX)

Modify the order price of OrderRef: 30001 from 122.1 to 122.2

OrderRef: 300001

Qty: 2000

Price: 122.2

```

01 35 00 00 00 54 2E 81 D5 04 50 32 6B 41 6C 4D
69 47 20 20 20 20 20 20 20 20 20 20 31 30 30 33
E1 93 04 00 00 D0 ED 90 2E 00 00 00 00 67 5E D8
02 00 00 00
  
```

Reply data packet (HEX)

Order No: 300001

Status: 301

Order Qty: 2,000

Order Price: 122.200

Order OS Qty: 2,000

Order Filled Qty: 0

Filled Qty: 0

Filled Price: .000

Stock Code: 1

Side: B

AON: N

Order Type: EL

Message Type: Modify Approve

Broker ID: 3032

Txn Time: 20140425155118570

Exec ID:

Text:

```

01 FE 00 00 00 55 2E 81 D5 04 50 32 6B 41 6C 4D
69 47 20 20 20 20 20 20 20 20 20 20 31 30 30 32
E1 93 04 00 2D 01 00 00 00 D0 ED 90 2E 00 00 00
00 67 5E D8 02 00 00 00 00 D0 ED 90 2E 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 01 00 00 00 01 45 00 02
33 30 33 32 00 00 00 00 00 00 00 00 32 30 31 34
30 34 32 35 31 35 35 31 31 38 35 37 30 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 01 00 00 00
  
```

The values highlighted with color are API header and trailer.

E_ORDERS_REQUEST_CANCEL – Cancel an existing order**API ID Number**

81101103

Description

Use this API to cancel the outstanding quantity of an existing order

Request Parameters

Name	Start	Length	Data Type	Description	Values (HEX)
OrderRef	0	4	32-bit integer	Order Ref of an order	

Reply

If the request passes basic validation, risk checking, the following single row order report message will be returned. Otherwise, error reply will be returned.

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer #	Order quantity	
Order Price	16	8	64-bit Integer #	Order price	
Order Outstanding Qty	24	8	64-bit Integer #	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer #	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer #	For Order Fill Report, it is the filled quantity of this report	
Filled Price	48	8	64-bit Integer #	For Order Fill Report, it is the filled price of this report	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Order Push Report	63	1	Byte	Indicator of this message	0x04 = Order Cancel Approve 0x05 = Order Cancel Reject
Broker ID	64	12	String	Broker ID of this order	Null terminated ASCII string
Transaction Date Time	76	25	String	The transaction date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	Always empty	null terminated ASCII string
Text	122	96	String	The reject reason	If rejected, contains a null terminated ASCII string reason
Report ID	218	4	32-bit Integer	This Report ID	

Remark

If there is no throttle rate available, an error reply will be returned;

Error Reply

Error Code	Description
81001	Insufficient fund
81002	Insufficient stock balance
81003	Order not found
81005	Out of throttle rate. The error reason contains the milliseconds to wait for next throttle
81006	System not ready

Cancel Order Example

Request data packet (HEX)

Cancel the order of OrderRef: 30001

OrderRef: 300001

01 25 00 00 00 54 2F 81 D5 04 50 32 6B 41 6C 4D

69 47 20 20 20 20 20 20 20 20 20 20 31 30 30 34

E1 93 04 00

Reply data packet (HEX)

Order Ref: 300001

Status: 402

Order Qty: 2,000

Order Price: 122.200

Order OS Qty: 0

Order Filled Qty: 0

Filled Qty: 0

Filled Price: .000

Stock Code: 1

Side: B

AON: N

Order Type: EL

Message Type: Cancel Approve

Broker ID: 3032

Txn Time: 20140425155123010

Exec ID:

Text:

01	FE 00 00 00	55	2F 81 D5 04	50 32 6B 41 6C 4D
69 47	20 20 20 20 20 20 20 20 20 20 31 30 30 32			
E1 93	04 00 92 01 00 00 00 00 D0 ED 90 2E 00 00 00 00			
00 67	5E D8 02 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 01 00 00 00 01 45 00 04			
33 30	33 32 00 00 00 00 00 00 00 00 00 00 32 30 31 34			
30 34	32 35 31 35 35 31 32 33 30 31 30 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00 00	00 00 00 00 00 00 00 00 00 00 01 00 00 00			

The values highlighted with color are API header and trailer.

E_ORDERS_REPORT_PUSH – Order report generate by system

API ID Number
81101999

Description

In receive of order fill, order cancel and trade cancel, system send out the following order type report.

Single Row Order Report

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer #	Order quantity	
Order Price	16	8	64-bit Integer #	Order price	
Order Outstanding Qty	24	8	64-bit Integer #	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer #	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer #	For Order Fill Report, it is the filled quantity of this report	
Filled Price	48	8	64-bit Integer #	For Order Fill Report, it is the filled price of this report	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Order Push Report	63	1	Byte	Order Push Report Type. Indicator of this message	0x00 = Order Report 0x01 = Order Fill 0x02 = Order Modify Approve 0x03 = Order Modify Reject 0x04 = Order Cancel Approve 0x05 = Order Cancel Reject 0x06 = Unsolicited Cancel 0x07 = Trade Cancel
Broker ID	64	12	String	For Order Report, it is the broker ID of the order. For Order Fill Report, it is the counterparty broker ID	Null terminated ASCII string
Transaction Date Time	76	25	String	The transaction date time in YYYYMMDDHHmmssfff format	Null terminated ASCII string
Execution ID	101	21	String	For order fill report type, an unique Execution ID from HKEx	null terminated ASCII string
Text	122	96	String	The reject reason of the order or	Null terminated ASCII string

				order modification	
Report ID	218	4	32-bit Integer	This Report ID	

Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.

E_ORDERS_REPORT_LIST – List out the order reports

API ID Number

81101104

Description

Use this API to get the order reports generated by the system. Each order report contains a unique ReportRef field, which is a subsequent number generated by system.

Request Parameters

Name	Start	Length	Data Type	Description	Values (HEX)
Start	0	4	32-bit Integer	Starting OrderRef	
End	4	4	34-bit Integer	Ending OrderRef	

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer	Order quantity	Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.
Order Price	16	8	64-bit Integer	Order price	
Order Outstanding Qty	24	8	64-bit Integer	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer	Always zero	
Filled Price	48	8	64-bit Integer	Always zero	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Report Type	63	1	Byte	Indicator of this message	0x00 = Order Report 0x01 = Order Fill 0x02 = Order Modify Approve 0x03 = Order Modify Reject 0x04 = Order Cancel Approve 0x05 = Order Cancel Reject 0x06 = Unsolicited Cancel 0x07 = Trade Cancel
Broker ID	64	12	String	Broker ID for this order	Null terminated ASCII string

Transaction Date Time	76	25	String	The transaction date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	For order fill report type, an unique Execution ID from HKEx	null terminated ASCII string
Text	122	96	String	The reject reason	If rejected, contains a null terminated ASCII string reason
Report ID	218	4	32-bit Integer	This report ID	

E_ORDERS_LISTALL – List out all the orders**API ID Number**

81101105

Description

Use this API to get the all orders

Request Parameters

No request parameter

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer	Order quantity	Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.
Order Price	16	8	64-bit Integer	Order price	
Order Outstanding Qty	24	8	64-bit Integer	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer	Always zero	
Filled Price	48	8	64-bit Integer	Always zero	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Report Type	63	1	Byte	Indicator of this message	0x00 = Order Report
Broker ID	64	12	String	Broker ID for this order	Null terminated ASCII string
Transaction Date Time	76	25	String	The order input date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	Always blank	
Text	122	96	String	The reject reason, if the order is rejected	If rejected, contains a null terminated ASCII string reason
Report ID	218	4	32-bit Integer	Always zero	

E_ORDERS_FILL_LISTALL – List out all the orders**API ID Number**

81101106

Description

Use this API to get the all the order fill records.

Request Parameters

No request parameter

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Status	4	4	32-bit Integer	Status of an order. Refer to orders status document	
Order Qty	8	8	64-bit Integer	Order quantity	Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.
Order Price	16	8	64-bit Integer	Order price	
Order Outstanding Qty	24	8	64-bit Integer	The outstanding quantity of an order	
Order Filled Qty	32	8	64-bit Integer	Total filled quantity of an order	
Filled Qty	40	8	64-bit Integer	Filled quantity	
Filled Price	48	8	64-bit Integer	Filled price	
Stock Code	56	4	32-bit integer	HKEx Numeric Stock code	
Order Side	60	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Order Type	61	1	Byte	One of the followings: AL = Auction Limit order AO = Auction order SL = Special Limited order EL = Enhance Limit order PL = Limit order	0x41 = AL 0x4F = AO 0x53 = SL 0x45 = EL 0x50 = PL
AON	62	1	Byte	All or nothing indicator	0x00 = Not set AON 0x01 = Set AON
Report Type	63	1	Byte	Indicator of this message	0x01 = Order Fill
Broker ID	64	12	String	Counterparty broker ID	Null terminated ASCII string
Transaction Date Time	76	25	String	The fill date time	Null terminated ASCII string in YYYYMMDDHHmmssfff format
Execution ID	101	21	String	For order fill report type, an unique Execution ID from HKEx	Null terminated ASCII string
Text	122	96	String	Always blank	
Report ID	218	4	32-bit Integer	Always zero	

E_ORDERS_OUTSTANDING_LISTALL – List out all outstanding orders**API ID Number**

81101109

Description

Use this API to get the all the outstanding orders

Request Parameters

No request parameter

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description	
OrderRef	0	4	32-bit Integer	Unique identifier for an order	
Order Price	4	8	64-bit Integer	Order Price	Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.
Order Outstanding Qty	12	8	64-bit Integer	The outstanding quantity of an order	
Stock Code	20	4	32-bit integer	HKEx Numeric Stock code	
Order Side	24	1	Byte	Buy or Sell or Short selling	0x1 = Buy 0x2 = Sell 0x5 = Short selling
Reference	25	14	Byte	The last successful Reference field passed-in by API caller	

E_CASH_BALANCE – Get the cash balances

API ID Number

81101107

Description

Use this API to get balance of the account. Each row represents a balance for one currency.

Request Parameters

No request parameter

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description
Opening Ledger Balance	0	8	64-bit Integer #	Opening ledger balance
Opening On Hand Balance	8	8	64-bit Integer #	Opening on hand balance
Current Ledger Balance	16	8	64-bit Integer #	Current ledger balance
Current On Hand Balance	24	8	64-bit Integer #	Current on hand balance
Currency	32	4	String	Currency code in null terminated string. Currently, there are following known value, more may be added to it later. • HKD • USD • CNY • JPY • EUR

Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.

E_INSTRUMENT_BALANCE – Get the instrument balances

API ID Number

81101108

Description

Use this API to get balance of the account. Each row represents a balance for one currency.

Request Parameters

No request parameter

Reply

The following multiple rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description
Opening Ledger Balance	0	8	64-bit Integer #	Opening ledger balance
Opening On Hand Balance	8	8	64-bit Integer #	Opening on hand balance
Current Ledger Balance	16	8	64-bit Integer #	Current ledger balance
Current On Hand Balance	24	8	64-bit Integer #	Current on hand balance
Stock code	32	4	32-bit Integer	HKEx numeric stock code

Signed 64 bit integer field with 8 implied decimal place. For example, for value 10000.03100012, it shall be multiple this value by 100000000 and converts it to an integer. The resulting 64 bit integer shall be 1000003100012.

E_HEARTBEAT – Heart beat to system

API ID Number

81102103

Description

Use this API to send heartbeat message to the system. The heartbeat is optional. The system will not disconnect the client even if no heartbeat is received from client.

Request Parameters

No request parameter

Reply

No reply parameter. System will send you back the API header only.

E_GET_TIME – Get local time

API ID Number

81102104

Description

Use this API to get machine time of the server.

Request Parameters

No request parameter

Reply

The following single rows order report message will be returned.

Order Report Reply

Name	Start	Length	Data Type	Description
tm_sec	0	4	32-bit Integer	Seconds after the minute (range 0-61) tm_sec is generally 0-59. The extra range is to accommodate for <i>leap seconds</i> in certain systems.
tm_min	4	4	32-bit Integer	Minutes after the hour (range 0-59)
tm_hour	8	4	32-bit Integer	Hours since midnight (range 0-23)
tm_mday	12	4	32-bit Integer	Day of the month (range 1-31)
tm_mon	16	4	32-bit Integer	Months since January (range 1-12)
tm_year	20	4	32-bit Integer	Years since 1900
tm_wday	24	4	32-bit Integer	Days since Sunday (range 0-6)
tm_yday	28	4	32-bit Integer	Days since January 1 (range 0-365)
tm_isdat	32	4	32-bit Integer	Daylight saving time flag. This flag is greater than zero if Daylight Saving Time is in effect, zero if Daylight Saving Time is not in effect, and less than zero if the information is not available.

The above reply fields are value provided by standard C++ “struct tm” data structure.

The value is get by the following C++ method:

```
#include <time.h>

time_t rawtime;
struct tm * timeinfo;
time (&rawtime);
timeinfo = localtime (&rawtime);
```

Library function Definition

StartTopTraderFlash function

Description

Start Top Trader Flash.

Syntax

```
void StartTopTraderFlash ();
```

Parameters

None

Return value

None

Remark

When this function is called, the library startup itself. It makes a TCP connection with TopTraderCore system and loads various data from database. It remains connect with TopTraderCore all the time.

The library also makes a TCP connection with HKEx; logon to the HKEx OCG session. It remains connect with HKEx OCG session all the time.

You must call this function before any other function.

And you can only call this function once.

StopTopTraderFlash function

Description

Shutdown Top Trader Flash.

Syntax

```
void StopTopTraderFlash ();
```

Parameters

None

Return value

None

Remark

When this function is called, the library logout and disconnect with HKEx OCG session. It also disconnect to TopTraderCore system.

It is recommended, not a requirement, you call this function before close your application.

However, it is a requirement to quit your application every day.

BindAccount function

Description

Bind a connection with the library.

Syntax

```
int BindAccount (  
    TopTraderFlashAccountCallback pfnCallback  
);
```

Parameters

A callback function pointer of type "TopTraderFlashAccountCallback"

Callback function

The callback function is defined as:

```
typedef void(*TopTraderFlashAccountCallback)(int, const char*, size_t);
```

Return value

If success, an interger connectionID to represent this binding.

If fail, -1.

Remark

Top Trader Flash support:

- Single account makes one connection
- Single account makes multiple connections (not recommend as it affect speed)
- Multiple accounts each make a separate connection
- Multiple account each make a multiple connections (not recommend as it affect speed)

Call this function to bind a connection with TopTraderFlash. The return value is a unique connectionID to represent this binding. A subsequent call to SentClientRequest takes this connectionID as first parameter. When the library involves the callback function, the first parameter is this connectionID.

This function only make reserve for the calling application. The actual binding between this connection and account need to be done when caller calls SendClientRequest function, passing-in E_LOGON binary data.

SendClientRequest function

Description

Make request to library

Syntax

```
int SendClientRequestAccount (  
    int connectionID,  
    const char* pData,  
    size_t len  
);
```

Parameters

Parameters	Description
connectionID	Connection ID obtains from BindAccount function
pData	byte array representing the requesting function as defined in this document
len	Length of the byte array

Return value

If success, echo back the connectionID

If fail, -1

Remark

Calling application shall pack the request as binary as documented on this document.

UnbindAccount function

Description

Unbind the connection

Syntax

```
void UnbindAccount (  
    int connectionID,  
);
```

Parameters

Parameters	Description
connectionID	Connection ID obtains from BindAccount function

Return value

None

Remark

Library release all resource from this connection.

TopTraderFlashAccountCallback function

Description

It is a callback function call by library

Syntax

The callback function shall be:

```
void OnCallback (  
    int connectionID,  
    const char* pData,  
    size_t len  
);
```

Parameters

Parameters	Description
connectionID	Connection ID obtains from BindAccount function
pData	The library packed the reply data as documented
len	Length of the pData

Return value

None